Modeling What Changes: Sparse, Residual World Models for Object-Centric Manipulation

Anonymous Author(s)
Submission under double-blind review

***Abstract*—Monolithic world models predict the entire next state at every step, spending capacity re-predicting the static majority of a scene and injecting error into it. We ask whether explicitly modeling change (a per-object change gate plus a residual delta head that perturbs only the objects the gate flags) is a more effective and interpretable bias for physical prediction and control. On a MuJoCo tabletop pushing benchmark scaling from 3 to 8 objects, the sparse/residual model predicts next-state poses 2.5 to 4.6 times more accurately than a dense multilayer perceptron at 8.6 to 11.1 times fewer parameters, sustains change-detection F1 of 0.80 to 0.87 where the dense baseline is degenerate, transfers across object counts with zero retraining (99.4 percent F1 retention), and reaches about 90 percent of its full-data accuracy with a quarter of the data. In autoregressive rollout it compounds far less error, hugging the no-motion floor while the dense model drifts. Finally, inside a sampling-based planner, prediction-only models fail (though a true-simulator oracle solves the task with the identical planner, confirming the planner is sound), but once featurized and trained for the states a planner visits, the sparse model *begins* to plan (0.23 plus or minus 0.06 success over three seeds) while the dense monolith stays at zero at every seed. Modeling *what changes*, rather than re-predicting the whole world, is a simple, effective bias for object-centric physical AI; code, data generators, and all checkpoints will be released upon publication.**

***Index Terms*—world models, object-centric learning, model-based planning, manipulation, conditional computation**

I. Introduction

Physical environments are overwhelmingly sparse in *change*. When a robot nudges one block on a cluttered table, a single object's pose changes and the rest of the scene is exactly as it was. Yet the dominant recipe for learned world models, encoding the scene and predicting the entire next state (or latent) monolithically [1], [2], ignores this structure. Such a model must reproduce every object at every step, which has two costs. First, *compute and capacity*: parameters and operations are spent re-predicting the static majority rather than the few objects that matter. Second, and more insidious, *error injection*: a regressor that outputs all poses inevitably perturbs objects that should have been copied verbatim, and in a closed-loop rollout that error accumulates on precisely the objects that never moved, so the predicted world drifts.

There is also an *interpretability* gap: a monolithic predictor offers no explicit handle on *what* it thinks changed, entangling that belief in a single regression output. For control and debugging, a model that names the objects it expects to move is far more useful than one that silently re-renders everything.

We ask whether explicitly modeling what changes addresses all three. Our model is deliberately simple and object-centric: a per-object change *gate* predicts which objects move, and a residual *delta head* predicts a pose increment applied *only* to gated objects; every other object is carried forward unchanged (Fig. 1). This sparse/residual factorization mirrors the sparsity of physical interaction, shares parameters across objects, is independent of object count, and exposes an explicit, inspectable change mask.

We evaluate on a procedurally generated MuJoCo tabletop pushing benchmark scaling from 3 to 8 free objects, against a dense multilayer perceptron (MLP) state predictor and a no-op (predict-no-change) baseline. Our contributions:

- A sparse/residual object-centric world model (gate plus residual delta head) that predicts tabletop dynamics 2.5 to 4.6 times more accurately than a dense MLP at 8.6 to 11.1 times fewer parameters, with the advantage concentrated on detecting and localizing the sparse set of objects that move (Section V).
- Analyses isolating *why* it wins: a parameter-matched control removing the "sparse is just smaller" confound, a capacity-matched ablation ladder showing the *gate*, not object-centricity alone, is the largest single contributor, and an oracle-gate diagnostic showing the residual bottleneck is delta *regression*, not change detection (Section VI).
- Evidence the prior is what makes it usable as a *world* model: it compounds far less error over multi-step rollout, transfers across object counts with zero retraining, and is markedly more sample efficient (Section VII).
- A downstream planning study (Section VIII): prediction-trained models cannot plan, but a true-simulator oracle can with the identical planner; and once featurized and trained for the planner's state distribution, the sparse model *begins* to plan while the dense monolith stays at zero at every seed; the object-centric advantage extends from prediction to control.

Scope. A deliberately controlled study, whose bounds we state up front: structured per-object state (poses and velocities), not pixels; 3 to 8 rigid boxes; small MLPs (< 0.1M parameters) that train, plan, and evaluate on a single laptop; and one push-to-goal planning task. Section IX revisits the consequences.

Code, data generators, checkpoints, and machine-readable result tables will be released upon publication.

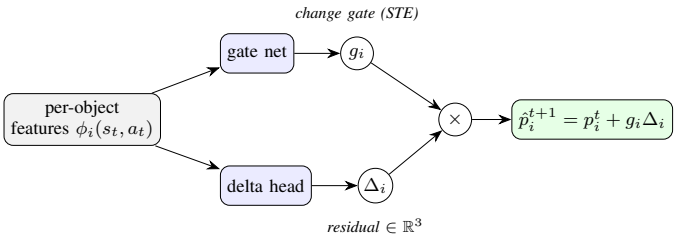

Fig. 1. The sparse/residual head, applied per object with weights shared across objects. A gate network emits a change decision g_i (discrete, trained by a straight-through estimator); a delta head emits a residual pose increment Δ_i. The next pose is the current pose plus the *masked* residual, so objects the gate leaves off are copied verbatim.

II. Related Work

Dense world models and model-based control. Learned world models compress observations into a latent state and predict its evolution for imagination, planning, and policy learning [1], [2]; sampling-based MPC (PETS [3], MPPI [4], CEM [5]) rolls action sequences through such a model. These predict the full state or latent *monolithically*, with no notion of which parts of the scene are inert, so error spreads across the whole state.

Object-centric and structured world models. Graph- and object-factored dynamics models share computation across entities and generalize across entity counts [6]–[8]. Ours is in this family (per-object features, shared weights, count-agnostic) but differs in *what* it predicts: an explicit per-object change *decision* plus a residual, targeting the sparsity of interaction, not only its relational structure.

Sparse coding and conditional computation. Good representations of natural signals are *sparse* [9]. Conditional-computation and sparsely-gated architectures route each input through a small, input-dependent subset of the network via learned discrete gates [10], [11], trained with straight-through or relaxed-categorical estimators [12], [13]. We import this into the *output* of a dynamics model, making sparsity a property of the prediction itself rather than of intermediate features.

Pushing dynamics. Pushing spans analytic mechanics [14], real-world planar-pushing datasets [15], and learned forward/inverse models [16], [17]. Closest to ours, SE3-Nets [18] predict per-object rigid-body motions with soft masks from point clouds; we instead learn a *discrete* change gate under an explicit sparsity objective. **The gap.** Object-centric models supply relational structure but still predict every object densely; conditional-computation methods supply input-dependent sparsity but not over a world model's *change structure*. We close that gap and evaluate the full chain from prediction to control.

III. Method

State and action. The scene state s_t concatenates the pusher position, per-object planar pose (x, y, θ), per-object velocity, and the goal. Actions a_t are end-effector delta-xy commands to a position-controlled pusher.

Dense baseline. A multilayer perceptron $f_\theta(s_t, a_t)$ regresses all object poses at $t{+}1$ under an L_2 loss (hidden width 256, 3 layers).

Sparse/residual model (Fig. 1). For each object i we build a feature vector $\phi_i(s_t, a_t)$ from its pose, velocity, relative goal and pusher positions, the action, and a permutation-invariant aggregate over the other objects. A gate network outputs a change logit; a delta head outputs a residual $\Delta_i \in \mathbb{R}^3$. The predicted next pose is the current pose plus the *masked* residual,

$$\hat{p}_i^{\,t+1} = p_i^{\,t} + g_i\,\Delta_i, \qquad g_i \sim \mathrm{Gate}(\phi_i), \tag{1}$$

where $g_i \in \{0, 1\}$ is a discrete gate sampled with a Gumbel straight-through estimator [11], [12] so the model trains end-to-end while remaining hard (exactly zero update) at inference. Because the gate and delta heads are shared across objects and no layer is sized to the object count, a model trained at one count runs at any other.

Loss. We combine a class-balanced binary cross-entropy on the gate against the ground-truth changed mask $m^\star$, an L_2 regression on the residual supervised only on changed objects, and a sparsity penalty on the mean gate activation $\bar{g}$:

$$\mathcal{L} = \mathrm{BCE}(g, m^\star) + \lambda_\Delta\, m^\star \cdot \|\Delta - \Delta^\star\|_2^2 + \lambda_s\, \bar{g}. \tag{2}$$

The sparsity weight λ_s trades gate recall for precision: a five-point sweep shows precision rising 0.94 to 0.97 and recall falling 0.81 to 0.71 as λ_s grows, with F1 degrading only gently. We use $\lambda_s{=}0.2$ for prediction.

IV. Experimental Setup

Environment and data. A procedural MuJoCo tabletop with N free 5 cm boxes and a scripted pushing policy. For each (N, seed) we log (s_t, a_t, s_{t+1}) with ground-truth per-object changed mask and delta, filter to a hard subset (steps with real motion) so metrics are not dominated by trivially static steps, and split 80/10/10 with an explicit configuration-leakage guard. **Baselines.** Dense MLP; no-op (predict no change). **Metrics.** Overall and changed/unchanged per-object pose L_2; change-detection precision, recall, and F1; parameters, operations, latency. **Scaling.** $N \in \{3, 5, 8\}$; the 8-object layout uses wider bounds and tighter spacing to fit the boxes (a stated caveat for cross-N density trends). Tables I–IV are mean plus or minus standard deviation over three training seeds; single-seed (seed 0) analyses are flagged in place. **Hardware.** All models are small MLPs (< 0.1M parameters) and run on a single laptop (Intel Core i7-12650H CPU, NVIDIA RTX 3050 GPU, CUDA 12.6); either device works via `--device`. Latencies are CPU: at this scale per-call GPU kernel-launch overhead dominates. **Training.** Adam, learning rate 10^{-3}, batch 128; 25 epochs dense, 15 sparse (25 for the contact/planning variant). Gate and delta heads are each 2-layer width-128 MLPs; Gumbel temperature 1.0, delta term weighted by the predicted gate probability, $\lambda_s{=}0.05$ for planning. Data: 250 scripted episodes $\times$ 100 steps per (N, seed), hard-subset threshold 0.02 m; the planning set adds 200 scripted $\times$ 80 and 350 random $\times$ 60 episodes. CEM: elite fraction 0.1, initial std 0.6, terminal weight 3.0, proximity weight 0.3, $\leq$60 steps per episode.

TABLE I
PREDICTION ACCURACY AND EFFICIENCY (MEAN ± STD, 3 SEEDS).

N	sparse F1	sparse L_2	dense L_2	params (dense/sparse)
3	0.867 ± 0.021	0.136 ± 0.046	0.347 ± 0.053	11.1×
5	0.802 ± 0.024	0.101 ± 0.004	0.319 ± 0.007	9.8×
8	0.829 ± 0.008	0.071 ± 0.016	0.329 ± 0.023	8.6×

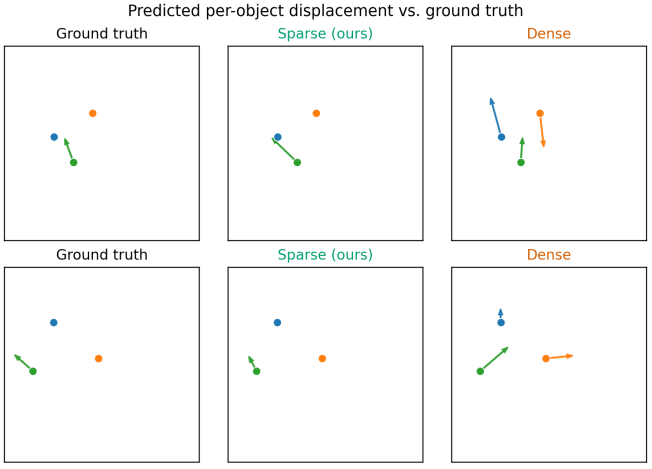

Fig. 2. Predicted per-object displacement (arrows from current to predicted next position) on two example scenes. The sparse model (center) moves only the object that truly moves, matching ground truth (left); the dense model (right) hallucinates motion on objects that should stay put.

V. PREDICTION ACCURACY

Table I reports the headline comparison. The sparse model's overall per-object L_2 is 2.5 to 4.6 times lower than the dense baseline at every object count, with non-overlapping error bars, at 8.6 to 11.1 times fewer parameters. The durable advantage over the no-op baseline is *change detection*: the sparse gate holds F1 0.80 to 0.87 and precision 0.92 to 0.97, while the dense model's change detection is degenerate (recall about 1: it flags every object as changed). Fig. 2 shows this qualitatively: the sparse model moves only the object that moves and copies the rest, whereas the dense model perturbs the whole scene. On overall L_2 the margin over no-op shrinks as scenes grow sparser with larger N, which is why we foreground change-detection F1 and changed-object L_2 as primary; a dense-interaction control (objects packed so pushes cascade) restores an about 12 times margin at $N{=}8$.

Efficiency, honestly. The win is *parameters*, not wall-clock: per-object heads scale with N, eroding the operation-count ratio (3.8× to 1.1×), and dense's single matmul is faster in latency. We claim capacity efficiency, not speed.

VI. WHY IT WINS

(All diagnostics in this section are seed 0, test split.) **Not just smaller.** Shrinking the dense MLP to the sparse parameter budget does not help it: at matched parameters sparse's overall L_2 is 3 to 5 times lower and its F1 far higher at every count, so the degeneracy is not a capacity artifact.

TABLE II
CAPACITY-MATCHED LADDER: EACH RUNG ADDS ONE INGREDIENT. OVERALL PER-OBJECT L_2; THE THREE UNGATED RUNGS SHARE ONE F1 COLUMN BECAUSE THEIR CHANGE DETECTION IS *identical*.

	overall L_2					F1	
N	dense	OC-ABS	OC-RES	**sparse**	no-op	ungated	**sparse**
3	0.367	0.239	0.221	**0.116**	0.128	0.538	**0.885**
5	0.318	0.194	0.171	**0.104**	0.107	0.388	**0.777**
8	0.354	0.187	0.159	**0.081**	0.081	0.294	**0.837**

It is the gate, not just object-centricity. The sparse model is both object-centric *and* change-modeling, so the headline gap cannot say which ingredient earns the win. A capacity-matched ladder (Table II) adds one at a time: a shared-weight per-object MLP on the *same* features predicting absolute poses (OC-ABS), the same predicting an always-applied residual (OC-RES), then the gated model; the OC rungs match the sparse parameter count to within 0.04%. Object-centric featurization earns 35 to 47 percent of the L_2 gap and the residual a little more, but *the gate is the largest single step* (1.7 to 2.0 times) and the only one reaching below the no-op floor. Tellingly, all three *ungated* rungs share identical change detection (recall exactly 1), so the degeneracy belongs to ungated regression, which object-centric structure alone does not cure. The unchanged-object column shows the mechanism: $0.211 \to 0.111 \to 0.086 \to \mathbf{0.001}$ at $N{=}3$.

The bottleneck is regression, not detection. Feeding the delta head the *ground-truth* changed mask (an oracle gate) barely moves changed-object L_2: perfect detection shaves almost nothing off. So the sparse model wins by detecting what changed and *not* injecting error into the unchanged majority, not by superior changed-object regression; one-step contact-driven deltas (especially rotation) are simply hard. This points at the delta head as the highest-leverage place to improve.

VII. IT IS A WORLD MODEL, NOT JUST A REGRESSOR

Rollout (Table III, Fig. 3a). Closing the loop (predict, write back, repeat), the dense baseline perturbs every object every step and drifts, while the sparse gate copies unchanged objects verbatim and hugs the no-op reference while still tracking the movers. On a *strictly held-out* trajectory set (fresh episodes, unseen generation seed, verified disjoint from training), sparse is 3.4 to 6.4 times better than dense at horizon 20, at every count and every seed. **Compositional generalization (Fig. 3b).** With a count-invariant featurization, a sparse model trained on N-object scenes runs on M-object scenes with zero retraining: off-diagonal (transfer) F1 is 0.827 vs. 0.832 on the diagonal (99.4 percent retention). The dense monolith cannot even be *run* off-count. **Sample efficiency (Fig. 3c).** Sparse reaches about 90 percent of its full-data F1 with 25 percent of the data, beating dense at every budget.

VIII. DOWNSTREAM PLANNING

Each model serves as the forward simulator in a receding-horizon planner (cross-entropy method [5], 256 samples times 3

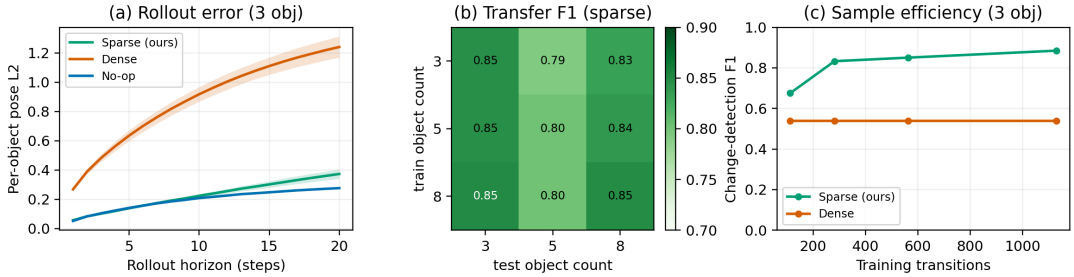

Fig. 3. The prior is what makes the model usable as a *world* model. (a) Rollout error vs. horizon (3 objects), held out, mean ± std (shaded) over 3 training seeds: the dense model drifts while the sparse model hugs the no-op "how much does the world move" floor. (b) Cross-count transfer F1 for the count-invariant sparse model; performance is set by the test count, not the training count (columns are near-uniform), and the dense monolith cannot be run off-count at all. (c) Sample efficiency (3 objects): the sparse model reaches about 90 percent of its full-data F1 with a quarter of the data, while the dense model's degenerate change detection is flat with data. Panels (b,c): seed 0, held-out test splits.

TABLE III
AUTOREGRESSIVE ROLLOUT ON THE HELD-OUT TRAJECTORY SET: PER-OBJECT POSE L_2 AT HORIZON 20 (MEAN ± STD, 3 TRAINING SEEDS; NO-OP IS SEED-INDEPENDENT).

N	sparse	dense	no-op	sparse vs. dense
3	0.373 ± 0.033	1.241 ± 0.072	0.277	3.4×
5	0.293 ± 0.038	1.097 ± 0.096	0.180	3.8×
8	0.182 ± 0.006	1.170 ± 0.022	0.108	6.4×

TABLE IV
PLANNING SUCCESS (MEAN ± STD, 3 SEEDS FOR LEARNED MODELS; SUCCESS RADIUS 5 CM).

condition	success	final dist. (m)
oracle (true simulator)	1.00	0.001
scripted (hand controller)	0.95	0.045
sparse (contact feats)	$\mathbf{0.23 \pm 0.06}$	$\mathbf{0.204 \pm 0.019}$
random	0.15	0.281
dense (diverse data)	$\mathbf{0.00 \pm 0.00}$	0.318 ± 0.021

refit iterations, horizon 15, replanning every step) on a push-the-target-to-the-goal task (3 objects, success is target within 5 cm of goal); the model is the only component swapped between conditions. A true-simulator *oracle* and a scripted controller are upper references, random actions the lower one.

Round 1: prediction-trained models cannot plan. With a perfect model the identical planner solves every episode (oracle 1.00, about 11 steps) and the task is clearly solvable (scripted 0.95), yet both learned models fail completely (0.00), worse than random. The cause is out-of-distribution querying: dense hallucinates 5 to 10 cm of motion with no contact, while sparse's velocity/context features leave its gate silent on the teleported, near-rest states a planner visits.

Round 2: fixing the diagnosed cause. We add a contact-aware, velocity-free feature mode (the pusher's post-action position, signed contact distance, and push direction, all well-defined for any queried state) and retrain on diverse mixed-policy data covering the approach angles the planner samples. With the planner unchanged (Table IV), sparse goes 0.00 to 0.23 plus or minus 0.06 success (three seeds) and halves its final-distance error, while dense stays at 0.00 at *every* seed: diverse data does not cure its hallucination. The object-centric advantage extends from prediction to control.

IX. LIMITATIONS

Scope: object-centric, not pixel-space. The model consumes and predicts *structured* state, not pixels: a deliberate boundary isolating whether modeling *change* helps from representation learning. Results therefore do not transfer directly to settings where poses must first be extracted from perception; pairing the change gate with an object-centric perceptual front end is future work. **Planning is preliminary, and narrow.** It tests a *single* planner (the cross-entropy method), a *single* push-to-goal task, and one cost at 3 objects; whether the advantage carries to gradient-based or MPPI planners, longer horizons, or multi-object goals is open. Sparse stays well below the scripted expert (0.23 vs. 0.95), landing *near* the goal but not reliably inside the 5 cm radius, and its margin over random, real on average, is not significant at the weakest seed; the robust claim is the separation from dense, which holds at every seed. **Other caveats.** The 8-object layout differs in geometry from 3 and 5, so cross-N density trends are not perfectly controlled; the efficiency claim is capacity, not wall-clock; and the Section VI analyses and Figs. 3b,c are single-seed, read as directional support for the three-seed results.

X. CONCLUSION AND FUTURE WORK

A per-object change gate plus a residual delta head predicts tabletop dynamics far more accurately than a dense monolith at a fraction of the parameters, compounds less error over rollout, transfers across object counts, is more sample efficient, and enables planning where the monolith cannot; the ablation ladder attributes the largest single share of that win to the gate itself. The oracle-gate diagnostic and the planning near-misses point at the same next step: a distributional delta head for multimodal contact, a DAgger loop [19] on planner-visited states, and multi-step rollout training; slot-based encoders [20] would drop the structured-state assumption. Modeling *what changes* is a simple, effective bias for object-centric physical AI.

REFERENCES

[1] D. Ha and J. Schmidhuber, "World models," *arXiv preprint arXiv:1803.10122*, 2018.

[2] D. Hafner, T. Lillicrap, J. Ba, and M. Norouzi, "Dream to control: Learning behaviors by latent imagination," in *International Conference on Learning Representations (ICLR)*, 2020.

[3] K. Chua, R. Calandra, R. McAllister, and S. Levine, "Deep reinforcement learning in a handful of trials using probabilistic dynamics models," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2018, pp. 4759–4770.

[4] G. Williams, N. Wagener, B. Goldfain, P. Drews, J. M. Rehg, B. Boots, and E. A. Theodorou, "Information theoretic mpc for model-based reinforcement learning," in *IEEE International Conference on Robotics and Automation (ICRA)*, 2017, pp. 1714–1721.

[5] R. Y. Rubinstein, "The cross-entropy method for combinatorial and continuous optimization," *Methodology and Computing in Applied Probability*, vol. 1, no. 2, pp. 127–190, 1999.

[6] P. W. Battaglia, R. Pascanu, M. Lai, D. Rezende, and K. Kavukcuoglu, "Interaction networks for learning about objects, relations and physics," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2016.

[7] A. Sanchez-Gonzalez, J. Godwin, T. Pfaff, R. Ying, J. Leskovec, and P. W. Battaglia, "Learning to simulate complex physics with graph networks," in *International Conference on Machine Learning (ICML)*, vol. 119, 2020, pp. 8459–8468.

[8] T. Kipf, E. van der Pol, and M. Welling, "Contrastive learning of structured world models," in *International Conference on Learning Representations (ICLR)*, 2020.

[9] B. A. Olshausen and D. J. Field, "Emergence of simple-cell receptive field properties by learning a sparse code for natural images," *Nature*, vol. 381, no. 6583, pp. 607–609, 1996.

[10] N. Shazeer, A. Mirhoseini, K. Maziarz, A. Davis, Q. Le, G. Hinton, and J. Dean, "Outrageously large neural networks: The sparsely-gated mixture-of-experts layer," in *International Conference on Learning Representations (ICLR)*, 2017.

[11] Y. Bengio, N. Léonard, and A. Courville, "Estimating or propagating gradients through stochastic neurons for conditional computation," *arXiv preprint arXiv:1308.3432*, 2013.

[12] E. Jang, S. Gu, and B. Poole, "Categorical reparameterization with gumbel-softmax," in *International Conference on Learning Representations (ICLR)*, 2017.

[13] C. J. Maddison, A. Mnih, and Y. W. Teh, "The concrete distribution: A continuous relaxation of discrete random variables," in *International Conference on Learning Representations (ICLR)*, 2017.

[14] M. T. Mason, "Mechanics and planning of manipulator pushing operations," *The International Journal of Robotics Research*, vol. 5, no. 3, pp. 53–71, 1986.

[15] K.-T. Yu, M. Bauza, N. Fazeli, and A. Rodriguez, "More than a million ways to be pushed: A high-fidelity experimental dataset of planar pushing," in *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2016, pp. 30–37.

[16] P. Agrawal, A. Nair, P. Abbeel, J. Malik, and S. Levine, "Learning to poke by poking: Experiential learning of intuitive physics," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2016.

[17] C. Finn and S. Levine, "Deep visual foresight for planning robot motion," in *IEEE International Conference on Robotics and Automation (ICRA)*, 2017.

[18] A. Byravan and D. Fox, "Se3-nets: Learning rigid body motion using deep neural networks," in *IEEE International Conference on Robotics and Automation (ICRA)*, 2017.

[19] S. Ross, G. J. Gordon, and J. A. Bagnell, "A reduction of imitation learning and structured prediction to no-regret online learning," in *International Conference on Artificial Intelligence and Statistics (AISTATS)*, 2011.

[20] F. Locatello, D. Weissenborn, T. Unterthiner, A. Mahendran, G. Heigold, J. Uszkoreit, A. Dosovitskiy, and T. Kipf, "Object-centric learning with slot attention," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2020.

APPENDIX A
HYPERPARAMETERS AND TRAINING

Dense baseline: MLP, hidden width 256, 3 layers, ReLU; L_2 loss on all poses; Adam, learning rate 10^{-3}, batch 128, 25 epochs. **Sparse/residual:** per-object features; gate and delta heads each a 2-layer MLP of width 128; Gumbel straight-through gate, temperature 1.0; loss (2) with the delta term weighted by the predicted gate probability; sparsity weight λ_s=0.2 (prediction) or 0.05 (planning); Adam, 10^{-3}, batch 128, 15 epochs (prediction) or 25 (contact/planning). **Data:** 250 scripted episodes ($\times$100 steps) per (N, seed) for prediction; a mixed set of 200 scripted ($\times$80) + 350 random ($\times$60) episodes for planning; hard-subset threshold 0.02 m; 80/10/10 split with a configuration-leakage guard. **Planner:** CEM, 256 samples, 3 iterations, elite fraction 0.1, horizon 15, initial std 0.6, terminal weight 3.0, proximity weight 0.3, replanning every step, up to 60 environment steps per episode.

APPENDIX B
METRIC AND PLANNER DEFINITIONS

Notation. A test split of N transitions with K objects each; $p_{n,i} \in \mathbb{R}^3$ is the planar pose (x, y, θ) of object i in transition n, with $\hat{p}$ the prediction and $p^\star$ the ground truth.

Pose error. Per-object error is the norm of the full pose residual,

$$e_{n,i} = \|\hat{p}_{n,i} - p^\star_{n,i}\|_2, \tag{3}$$

taken over all three components, so metres and radians are pooled in a single norm: a convention kept for comparability across models, not a claim that the units are commensurate. The three reported aggregates are the unweighted mean and the means restricted to changed and unchanged objects,

$$L_2^{\text{all}} = \frac{1}{NK}\sum_{n,i} e_{n,i}, \qquad L_2^{\text{ch}} = \frac{\sum_{n,i} m^\star_{n,i}\, e_{n,i}}{\sum_{n,i} m^\star_{n,i}}, \tag{4}$$

with L_2^{unch} defined likewise using $1 - m^\star$.

Change mask. The ground-truth mask thresholds the true displacement,

$$m^\star_{n,i} = \mathbf{1}\big[\|\Delta^\star_{xy}\|_2 > \epsilon_p\big] \;\vee\; \mathbf{1}[|\mathrm{wrap}(\Delta^\star_\theta)| > \epsilon_\theta], \tag{5}$$

with ϵ_p=10^{-3} m, ϵ_θ=10^{-2} rad, and wrap mapping angles to $(-\pi, \pi]$. The sparse model reports its gate g_i directly; every *ungated* baseline (dense, no-op, OC-ABS, OC-RES) has no change output, so its predicted mask comes from applying the *same* rule to its predicted delta. This is exactly why ungated recall is 1 in Table II: any nonzero regression output clears ϵ_p, so every object is flagged as changed. Precision, recall, and F1 are the standard quantities pooled over all NK object slots.

Planner. At each environment step the planner optimises an action sequence $a_{t:t+H-1}$ (H=15) by the cross-entropy method. Each of J=3 iterations draws B=256 candidates,

$$a^{(b)} \sim \mathcal{N}(\mu, \mathrm{diag}\,\sigma^2), \qquad a^{(b)} \leftarrow \mathrm{clamp}\Big(a^{(b)}, -1, 1\Big), \tag{6}$$

rolls each through the model under evaluation, keeps the $\lceil \rho B \rceil$ lowest-cost elites (ρ=0.1), and refits $\mu \leftarrow \mathrm{mean}(\text{elites})$, $\sigma \leftarrow \max(\mathrm{std}(\text{elites}), \sigma_{\min})$, starting from σ_0=0.6 with $\sigma_{\min}$=0.05. Writing $d_h = \|x_h^{\text{tgt}} - g\|_2$ for the imagined target-to-goal distance at horizon step h, the cost is

$$J = \frac{1}{H}\sum_{h=1}^{H} d_h \;+\; w_T\, d_H \;+\; \frac{w_p}{H}\sum_{h=1}^{H} \|x_h^{\text{pus}} - x_h^{\text{tgt}}\|_2, \tag{7}$$

with w_T=3.0 and w_p=0.3: the mean term supplies dense progress, the terminal term is the actual objective, and the proximity term makes contact discoverable. Only the first action is executed; the next step warm-starts μ from the shifted previous solution. An episode succeeds if $\|x^{\text{tgt}} - g\|_2 \le 5$ cm within 60 steps. Across all conditions only the model rolling the sequences forward differs.

APPENDIX C
FULL EFFICIENCY BREAKDOWN

Table V gives parameters, forward-pass operations, and CPU latency per model and object count (mean over 3 seeds). Parameter efficiency is the durable win; the operation-count advantage erodes with N as the per-object heads scale, and dense is faster in wall-clock latency (one matmul vs. per-object gating), reported here for transparency, not as a claim.

TABLE V
EFFICIENCY: PARAMETERS, OPERATIONS (FLOP ESTIMATE), CPU LATENCY.

N	sparse params	dense params	param ratio	sparse FLOPs	dense FLOPs	lat. (ms) sparse/dense
3	6 916	76 809	11.1×	39 936	152 576	0.19 / 0.04
5	8 452	82 959	9.8×	81 920	164 864	0.20 / 0.04
8	10 756	92 184	8.6×	167 936	183 296	0.20 / 0.04

APPENDIX D
SPARSITY-WEIGHT ABLATION

A five-point sweep of λ_s (3 objects, seed 0, otherwise identical config, Table VI) makes the gate more conservative (precision rises and recall falls) while pose error is essentially flat. The main runs use λ_s=0.2.

TABLE VI
EFFECT OF THE SPARSITY WEIGHT λ_s ON THE GATE.

λ_s	F1	precision	recall	changed-obj L_2	overall L_2
0.0	0.873	0.941	0.814	0.389	0.146
0.05	0.865	0.940	0.801	0.389	0.146
0.2	0.849	0.938	0.776	0.388	0.145
0.5	0.845	0.967	0.750	0.387	0.144
1.0	0.815	0.965	0.705	0.385	0.143

APPENDIX E
PARAMETER-MATCHED DENSE BASELINE

To remove the "sparse is just smaller" confound we shrink the dense MLP to the sparse model's exact parameter budget and retrain (Table VII, test split, seed 0). Sparse still wins on every metric at every count: shrinking dense does not help, and its change detection stays degenerate: the advantage is object-centric structure, not parameter count.

TABLE VII
SPARSE VS. PARAMETER-MATCHED DENSE (EQUAL BUDGET).

N	sparse L_2	dense-matched L_2	sparse F1	dense-matched F1
3	**0.116**	0.363	**0.885**	0.538
5	**0.104**	0.363	**0.777**	0.388
8	**0.081**	0.415	**0.837**	0.294

APPENDIX F
ORACLE-GATE DIAGNOSTIC

Feeding the delta head the ground-truth changed mask isolates detection from regression (Table VIII, changed-object L_2, test split, seed 0). Perfect detection barely moves the number, so the changed-object bottleneck is delta *regression*, not the gate.

TABLE VIII
CHANGED-OBJECT L_2: PREDICTED VS. ORACLE GATE VS. NO-OP.

N	predicted gate	oracle gate	no-op
3	0.312	0.314	0.348
5	0.426	0.432	0.446
8	0.466	0.474	0.470

APPENDIX G
PER-SEED PLANNING RESULTS

Table IX lists the per-seed planning numbers summarized in Section VIII. The sparse advantage over dense holds at every seed (dense is $0/20$ throughout); the sparse margin over random (0.15) holds on average but not at the weakest seed.

TABLE IX
CONTACT-AWARE PLANNING, PER TRAINING SEED (20 EPISODES EACH).

model	seed 0	seed 1	seed 2	mean $\pm$ std
sparse success	0.25	0.15	0.30	0.23 ± 0.06
dense success	0.00	0.00	0.00	0.00 ± 0.00
sparse fin. dist.	0.188	0.231	0.193	0.204 ± 0.019